

Relazione esercitazione incApache

- **Introduzione**

La relazione espone la metodologia utilizzata sia per creare il codice e sia per il debugging.

La versione inviata contiene il metodo 7.0 funzionante 7.1 funzionante più una bozza del metodo POST.

- **Obiettivo del laboratorio**

Il laboratorio incApache consiste nella creazione di un web server in c.

- **Metodologia utilizzata**

Partendo dalla funzione principale (`main`), abbiamo iniziato a esaminare tutte le funzioni già implementate seguendo la logica del codice. Durante questa fase di analisi, quando individuavamo eventuali lacune ci impegnavamo a colmarle attraverso il ricorso al manuale, video e l'approfondimento delle spiegazioni fornite in aula. Confrontandoci cercavamo di completare le parti mancanti con soluzioni ritenute più idonee e coerenti con il contesto del progetto. Questo approccio ci ha permesso non solo di migliorare la robustezza del codice, ma anche di assicurarci che ciascuna funzione fosse implementata in modo accurato e completo.

Con l'introduzione della versione 7.0, siamo riusciti a immergerci nel flusso del codice, comprendendone a pieno il funzionamento. La `send_response` è stata la funzione che ci ha richiesto più tempo, ma una volta completata e compresa, tutto è diventato più fluido e semplice.

Nel contesto della versione 7.1, la struttura dati ha costituito una sfida, poiché non è stato immediato comprendere come utilizzare l'array `to_join` per includere il thread precedente a quello appena creato.

- **to_join**

L'array `to_join` è stato popolato sfruttando l'indirizzamento diretto degli array.

Le prime quattro celle sono state riservate per contenere l'ultimo thread di risposta creato, mentre nella cella ancora non utilizzata dell'array (`new_thread_idx`) veniva inserito il thread precedente a quello appena aggiunto.

In questo modo, si assicura che nella posizione *i*-esima della struttura `to_join` ci sia il thread precedente a quello contenuto nella medesima posizione della struttura `thread_ids`.

Dopo aver popolato l'array e aver acquisito una comprensione più approfondita del funzionamento della struttura dati, ci siamo dedicati alle funzioni `join_prev` e `join_all`, che ci hanno permesso di riflettere sul concetto di thread e sul suo corretto utilizzo.

- **Problemi riscontrati**

Ciò che ha comportato un notevole dispendio di tempo è stata l'implementazione delle due funzioni di join. Inizialmente, la `join_prev` eseguiva la join di tutti i thread precedenti a quello passato come argomento. Tuttavia, una volta analizzata la posizione della chiamata, ci siamo resi conto che doveva effettuare la join solamente del thread precedente a quello specificato come argomento.

Inizialmente, la funzione `join_all` era stata implementata per attendere tutti i threads. Successivamente, ci siamo resi conto che ciascun thread risposta chiama la `join_prev` per aspettare il thread di risposta precedente prima di procedere; quindi la `join_all` doveva essere effettuata solo sull'ultimo thread risposta creato.

L'ultima complicazione è emersa durante la creazione della pagina dinamica, la quale continua a presentare delle problematiche che stiamo attualmente cercando di risolvere.

- **Debugging**

Nel processo di debug del codice, abbiamo frequentemente adottato il metodo di stampare alcuni dati essenziali per comprendere i valori che assumevano in punti specifici.

- **Completezza del codice**

Il progetto (per ora) garantisce il funzionamento corretto del web server sia in 7.0 che in 7.1.

Invece per quanto riguarda il metodo POST è presente, ma la pagina dinamica richiesta è ancora in fase di sviluppo.